PKM LLM

Guide d'installation et de mise en place

Installation · LLM Wiki · Claude Code + Obsidian

Système de connaissance persistant et auto-maintenu

Qu'est-ce que le LLM Wiki ?

Le LLM Wiki est un **Personal Knowledge Management system (PKM)** augmenté par les LLMs. Contrairement aux systèmes classiques comme NotebookLM ou les uploads de fichiers dans ChatGPT, où le LLM redécouvre la connaissance depuis zéro à chaque question, le LLM Wiki construit et maintient un **wiki persistant** qui s'enrichit à chaque source ajoutée.

Le principe est simple : tu apportes les sources et les questions, Claude Code fait tout le reste : résumer, croiser, mettre à jour les références, détecter les contradictions. Le wiki devient de plus en plus riche et dense au fil du temps, sans que tu aies à en écrire une seule ligne toi-même.

> *Ce système repose sur une idée d'Andrej Karpathy. Le concept original est disponible ici :*

llm-wiki — Andrej Karpathy

Le document fondateur du concept LLM Wiki · GitHub Gist

https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f

Initialisation

Les prérequis à installer avant de commencer

Logiciels requis

Avant de mettre en place le système, assure-toi d'avoir installé les quatre éléments suivants sur ta machine :

1. **Obsidian :** l'éditeur de notes qui servira d'interface visuelle pour ton wiki. C'est là que tu verras le graph view colorisé et que tu parcourras les pages générées par Claude Code.

2. **Visual Studio Code :** l'éditeur de code depuis lequel tu lanceras Claude Code. Obsidian et VS Code peuvent tourner côte à côte : Obsidian à droite pour naviguer le wiki, VS Code à gauche pour interagir avec Claude Code.

3. **Claude Code** (extension VS Code) **:** l'agent IA qui maintient ton wiki. Il s'installe directement depuis le marketplace d'extensions de Visual Studio Code en cherchant "Claude Code".

4. **Un vault Obsidian ouvert dans VS Code :** crée un vault Obsidian dans un dossier de ton choix, puis ouvre ce même dossier dans Visual Studio Code via Fichier → Ouvrir le dossier. Les deux applications pointent alors vers le même répertoire.

🚀 Installation du système

Initialiser le LLM Wiki dans ton vault Obsidian

Étape 1 — Lancer Claude Code

Ouvre Visual Studio Code avec ton vault Obsidian comme dossier racine. Lance ensuite Claude Code depuis le terminal intégré.

Étape 2 — Coller le prompt d'initialisation

Dans la console de Claude Code, colle le texte du **LLM Wiki** (le document de Karpathy ci-dessus) suivi de ce prompt d'instruction :

> *You are now my LLM Wiki agent. Implement this exact idea file as my complete second brain. Guide me step-by-step: create the CLAUDE.md schema file with full rules, set up index.md and log.md, define folder conventions, and show me the first ingest example. From now on, every interaction follows the schema.*

Claude Code va alors créer automatiquement toute la structure du wiki : le fichier `CLAUDE.md` (le schéma central), `index.md`, `log.md`, et les dossiers `wiki/`, `raw/`.

Installation des commandes

Créer les 10 fichiers de commandes slash dans Claude Code

Créer la structure de dossiers

Dans ton vault (le dossier racine ouvert dans VS Code), crée manuellement le dossier `.claude/`, puis à l'intérieur crée un sous-dossier `commands/`. Le chemin final est donc `.claude/commands/`.

Créer les 10 fichiers de commandes

Dans le dossier commands/, crée 10 fichiers markdown avec exactement ces noms :

- `back.md`
- `checkpoint.md`
- `explain.md`
- `ingest.md`
- `label.md`
- `lint.md`
- `query.md`
- `save.md`
- `synthesize.md`
- `update-context.md`

Remplir chaque fichier avec son prompt

Chaque fichier doit contenir le prompt correspondant à sa commande. Tous les prompts sont disponibles dans le dépôt GitHub suivant. Copie le contenu de chaque fichier et colle-le dans le fichier correspondant dans ton dossier `.claude/commands/` :

PKM-LLM-Prompt-French — max-leroy-gith

Dépôt GitHub contenant les 10 prompts de commandes en français

https://github.com/max-leroy-gith/PKM-LLM-French

> *Une fois les fichiers en place, redémarre Claude Code. Tape / dans la console pour vérifier que tes commandes apparaissent bien dans l'onglet custom-commands.*

Installer Git

Activer le versioning pour pouvoir utiliser /back

Git est le système de versioning qui permet à la commande `/back` de fonctionner. À chaque commande qui crée des fichiers (`/ingest`, `/save`, etc.), un commit Git est automatiquement créé. `/back` annule alors exactement ce dernier commit, rien de plus.

Initialisation, à faire une seule fois

Depuis un terminal normal (PowerShell ou terminal Windows, pas Claude Code), navigue vers le dossier de ton vault puis exécute ces commandes dans l'ordre :

```
# 1. Initialiser Git dans le dossier
git init

# 2. Ajouter tous les fichiers existants
git add .

# 3. Créer le premier commit
git commit -m "init: wiki initial"

# Si Git demande ton identité (erreur Author identity unknown) :
git config --global user.email "ton@email.com"
git config --global user.name "tonnom/pseudonyme"
# N'oublie pas d'y noter tes informations, en remplaçant « tonnom »

# Relancer le commit après avoir configuré l'identité
git commit -m "init: wiki initial"

# Vérifier que tout est bon
git log --oneline
# → doit afficher : xxxxxxx init: wiki initial
```

> *Git est maintenant opérationnel. À partir de ce moment, toutes tes commandes /ingest, /save, /synthesize, /explain, /label, /checkpoint et /update-context créeront automatiquement un commit Git à chaque exécution.*

Get started

Comment utiliser le système au quotidien

À chaque nouvelle session

Lance Claude Code depuis VS Code, dans le dossier de ton vault. Commence toujours par `/checkpoint` sans argument pour relire l'état de la dernière session et savoir exactement où tu en es avant de commencer.

Démarrer un nouveau projet

Lance la commande `/update-context` pour introduire ton projet au second cerveau. Claude Code te posera 5 questions pour créer le fichier `wiki/context/project-context.md`, la source de vérité centrale que toutes les autres commandes liront pour comprendre le contexte de ton projet.

Ajouter des sources avec le Web Clipper

Quand tu trouves un article, une documentation ou un contenu intéressant pour ton projet, utilise l'extension **Obsidian Web Clipper** pour le convertir en fichier markdown d'un seul clic. Le fichier atterrit directement dans ton dossier `raw/`. Tu peux ensuite lancer `/ingest raw/nom-du-fichier.md` pour l'intégrer dans le wiki.

Obsidian Web Clipper

Extension navigateur — capture et convertit les pages web en markdown en un clic

https://obsidian.md/clipper

> *Le Web Clipper est compatible avec Chrome, Firefox et Safari. Une fois installé, un clic sur l'icône depuis n'importe quelle page web envoie le contenu directement dans ton vault Obsidian au format markdown, prêt à être ingéré.*

Workflow recommandé

Routine type d'une session avec le second cerveau

Début de session

```
/checkpoint
→ Relit le dernier bilan : état du projet, prochaines étapes exactes
```

Pendant la session

```
# Ajouter une nouvelle source
/ingest raw/article-interessant.md

# Poser une question au wiki
/query "Comment fonctionne le tool calling ?"

# Sauvegarder une bonne réponse
/save nom-descriptif

# Comprendre un concept rencontré dans le code
/explain "rate limiting"

# Créer une synthèse complète sur un sujet
/synthesize "architecture multi-agents"
```

Fin de session

```
/checkpoint "Description de ce qui a été fait"
→ Crée un bilan, met à jour project-context.md, commit Git
```

Maintenance régulière (une fois par semaine)

```
/lint
→ Détecte contradictions, pages orphelines, gaps de contenu
→ Propose des sources à ingérer pour combler les manques

/label
→ Vérifie que toutes les pages ont les bons tags
→ Met à jour la colorisation du graph Obsidian
```